BLAST: Fast File Transfer over UDP

CSc 412 Computer Networks — Programming Project
Instructor: Prof. K. Ravindran
Student: Md Mamun (individual project)
Date: October 2026

1. Introduction

This project implements BLAST, a file-transfer protocol that sends a large file over UDP "in full blast": it sends many records back-to-back and asks for acknowledgment only once per blast, instead of acknowledging every packet as TCP does. Lost records are found by a single query at the end of each blast and resent selectively.

The implementation is in C using the UNIX socket interface to UDP. It consists of a file sender, a file receiver, and a garbler that drops packets at a set rate. I measured throughput for loss rates from 0% to 30%, for all three record sizes and three blast sizes, with 5 runs per setting (360 transfers in total). Every received file was checked against the original; all 360 transfers were correct.

The project was approved by the instructor to run on macOS instead of Linux. The code uses only POSIX sockets and threads and also builds on Linux.

2. System overview

File	Contents
`blast.h`	Packet formats shared by sender and receiver
`garble.h`	Garbler: drops each outgoing packet with a given probability
`sender.c`	File sender: protocol state machine, disk-reader thread, buffer B'
`receiver.c`	File receiver: protocol state machine, buffer B', disk-writer thread
`Makefile`	Builds `sender` and `receiver`
`run_experiments.sh`	Runs transfers over loss rates × record sizes × M, writes `results.csv`
`plot.py`	Produces the graphs and summary tables from `results.csv`

Build with `make`, then run:

```
./receiver <port> <max_M> [loss_rate]
./sender <receiver_ip> <port> <file> <record_size> <M> [loss_rate] [dest_name]
```

`record_size` is 256, 512 or 1024 bytes; `M` (records per blast) is 200–10000; `loss_rate` is a probability such as `0.1`; `dest_name` is the name the receiver stores the file under (default: the file's own name). The receiver serves one transfer after another until it is stopped.

Following Figure 1 of the assignment, each side has its own buffer B' between the disk and the UDP socket. On each side a separate thread moves data between the disk and B', so disk I/O and network I/O proceed asynchronously (Section 5).

3. Packet formats

All multi-byte fields are sent in network byte order.

Packet	Fields
FILE_HDR	type, file size, record size, M (proposed), name to store the file under
FILE_HDR_ACK	type, status (0 = accepted), record size, M (agreed)
DATA	type, N, N × (st_rec, end_rec), then the records of segments 1..N in order
IS_BLAST_OVER	type, seq, Mst, Mfin
REC_MISS	type, seq, Mst, Mfin, count, count × (st_rec, end_rec) — an empty list means the blast is complete
DISCONNECT	type
DISCONNECT_ACK	type

Records are numbered from 1. A DATA packet carries at most 16 records in at most 16 segments. On macOS a UDP datagram is limited to 9216 bytes by default, so 256- and 512-byte records travel up to 16 per packet and 1024-byte records up to 8 per packet. On Linux, 1024-byte records also travel 16 per packet.

M negotiation. The sender proposes M in FILE_HDR. The receiver accepts it or lowers it to its own buffer limit (`max_M`) and returns the agreed value in FILE_HDR_ACK. Both sides then use the agreed M and record size for the whole transfer.

4. Protocol state machines

4.1 Sender

State	Action	Event → next state
CONNECT	send FILE_HDR	FILE_HDR_ACK (accepted) → BLAST; timeout 1 s → resend (max 20 tries, then abort)
BLAST	send the records of the current blast as they become available from disk	all records sent → QUERY
QUERY	send IS_BLAST_OVER(Mst, Mfin) with a new seq	REC_MISS with ranges → RESEND; REC_MISS empty → next blast's BLAST, or DISCONNECT after the last blast; timeout 300 ms → resend IS_BLAST_OVER (max 30 tries, then abort)
RESEND	send the listed records, packed into multi-segment packets	all sent → QUERY
DISCONNECT	send DISCONNECT	DISCONNECT_ACK → DONE; timeout 300 ms → resend (max 10 tries, then DONE, since all data is already confirmed)

4.2 Receiver

State	Event	Action
IDLE	FILE_HDR	open `<name>.part`, agree on record size and M, start the writer thread, reply FILE_HDR_ACK → ACTIVE
ACTIVE	FILE_HDR (duplicate)	reply FILE_HDR_ACK again (the first ACK was lost)
ACTIVE	DATA	store new records in B', mark them received, hand the contiguous prefix to the writer
ACTIVE	IS_BLAST_OVER	reply REC_MISS listing missing ranges in Mst..Mfin, echoing seq
ACTIVE	DISCONNECT	flush B' to disk, close, rename `<name>.part` to `<name>`, reply DISCONNECT_ACK → LINGER
LINGER (3 s)	IS_BLAST_OVER / DISCONNECT	reply REC_MISS / DISCONNECT_ACK again (sender's reply was lost)
LINGER	3 s elapsed, or a new FILE_HDR	release the transfer → IDLE

4.3 Timeouts

Timeouts use `SO_RCVTIMEO` on the socket: `recvfrom` returns `EAGAIN` when no packet arrives within the timeout, and the sender retransmits. This behaves like `alarm()` with a SIGALRM handler, without the races of interrupting a system call from a signal handler.

4.4 Sequence number in IS_BLAST_OVER

I added a sequence number to IS_BLAST_OVER, which the receiver echoes in REC_MISS. Without it, a late REC_MISS answering an earlier query of the same blast could be mistaken for the answer to the current query, causing records to be sent again needlessly. The sender ignores any REC_MISS whose seq is not the current one.

4.5 Example

The trace below is from the timing scenario of Figure 2 in the assignment and matches what the implementation does (records 5–6, 8–10 and 21–38 lost in the first pass):

```
sender                                              receiver
{(1,4)} r1..r4 ------------------------------------> stored
{(5,6)} r5,r6  ---------------X lost
 ...
IS_BLAST_OVER(1,38) ------------------------------->
<---------------------------- REC_MISS({(5,6),(8,10),(21,38)})
{(5,6),(8,10),(21,31)} r5,r6,r8..r10,r21..r31 ----->  (3 segments in one packet)
{(32,38)} r32..r38 ------------X lost
IS_BLAST_OVER(1,38) ------------------------------->
<-------------------------------------- REC_MISS({(32,38)})
{(32,38)} r32..r38 ------------------------------->
IS_BLAST_OVER(1,38) ------------------------------->
<------------------------------------------------- REC_MISS({})
```

5. Buffers and asynchronous disk I/O

The assignment asks that reading from disk, network transfer, and writing to disk may proceed asynchronously, which is why a packet can carry a variable number of records.

Sender. B' is a ring of M record slots; record r lives in slot $(r-1) \bmod M$. A reader thread reads the file into B' in chunks of a random size between 1 and 40 records, independently of the network, and publishes how many records are available. The sending loop sends whatever records are available at that moment: if only 3 records are ready, it sends a packet with 3 records rather than waiting for 16. This reproduces the "records 5–6 available", "record 7 available" behaviour of Figure 2. The reader may refill a slot only after the blast that used it has been confirmed by an empty REC_MISS, because retransmissions still need the data.

Receiver. B' is also a ring of M record slots. The network side copies each new record into B' and keeps track of the longest prefix of the file that has fully arrived. A writer thread writes that prefix from B' to disk, independently of the network. The network side may reuse a slot only after the writer has written the record that was in it. On DISCONNECT the receiver waits for the writer to flush the rest, then closes the file and renames it from `<name>.part` to `<name>`, so a file under the final name is always complete.

The measurements show the variable packet sizes: with 256-byte records and M = 500, about 7% of the data packets carried fewer than 16 records (214 of 3037 on average at 0% loss), and the average was 15.4 records per packet.

6. Garbler

The garbler (`garble.h`) wraps `sendto`. Each outgoing packet is dropped with probability p, using an independent random draw. Both the sender and the receiver send through the garbler, so data packets and every control packet (FILE_HDR, FILE_HDR_ACK, IS_BLAST_OVER, REC_MISS, DISCONNECT, DISCONNECT_ACK) can be lost. Each program counts the packets it attempted to send and the packets the garbler dropped.

Table 1 shows that the measured drop rate matches the configured rate closely.

7. Experiment method

- **Machine:** Apple MacBook Pro 14-inch (2023, M2 Pro, 16 GB RAM), macOS 26.6.2.
- **Network:** sender and receiver on the same machine, communicating over the loopback interface (127.0.0.1).
- **File:** 12,000,021 bytes of random data (96.0 Mbit). The size is deliberately not a multiple of the record size, so the short final record is exercised.
- **Settings:** loss rate $p \in \{0, 1, 2, 5, 10, 15, 20, 30\}$%, record size $\in \{256, 512, 1024\}$ bytes, M $\in \{500, 2000, 10000\}$ records, 5 runs each: $8 \times 3 \times 3 \times 5 = 360$ transfers.
- **Correctness check:** after every transfer the received file was compared with the original by checksum (`cksum`). All 360 transfers were correct.
- **Throughput** = file size ÷ time from the start of the first blast to the empty REC_MISS for the last blast. Connection setup and disconnect are excluded.

Command used:

```
make
./run_experiments.sh test12mb.bin "256 512 1024" "500 2000 10000" 5
python3 plot.py
```

A note on the test setup. In the first full run, 4 of the 360 transfers were reported as failed. Investigation showed the received data was complete and byte-for-byte correct in every case: the project folder is synced by iCloud Drive, which occasionally renamed the receiver's output file

while it was being written (e.g. to `recv_test12mb 2.bin.part`), so the receiver's final rename failed. The experiment script now runs the receiver in a temporary directory outside the synced folder, and the 4 affected transfers were rerun under the same settings; all passed.

8. Results

8.1 Packet loss statistics

Table 1 is averaged over all 45 transfers (9 settings × 5 runs) at each loss rate. "Sender" counts packets sent by the sender (data, FILE_HDR, IS_BLAST_OVER, DISCONNECT); "receiver" counts packets sent by the receiver (ACKs and REC_MISS).

Table 1. Packets sent and lost per transfer (average)

Loss rate	Data packets sent	Sender: dropped / sent	Receiver: dropped / sent	Measured drop rate	Records resent	Timeouts
0%	1984	0 / 2010	0.0 / 25.1	0.00%	0	0.0
1%	2004	20 / 2041	0.5 / 35.7	1.01%	275	0.8
2%	2024	40 / 2067	0.7 / 40.8	1.92%	529	1.8
5%	2092	106 / 2148	2.8 / 52.4	4.94%	1423	5.4
10%	2210	229 / 2287	6.4 / 67.4	10.02%	3032	14.0
15%	2338	363 / 2437	12.2 / 83.2	14.89%	4788	26.0
20%	2480	515 / 2609	21.0 / 101.6	19.76%	6717	46.0
30%	2843	919 / 3060	44.4 / 148.6	30.03%	11683	110.4

Over all 360 transfers, the garbler dropped 102,623 of 864,598 packets.

8.2 Throughput versus loss rate

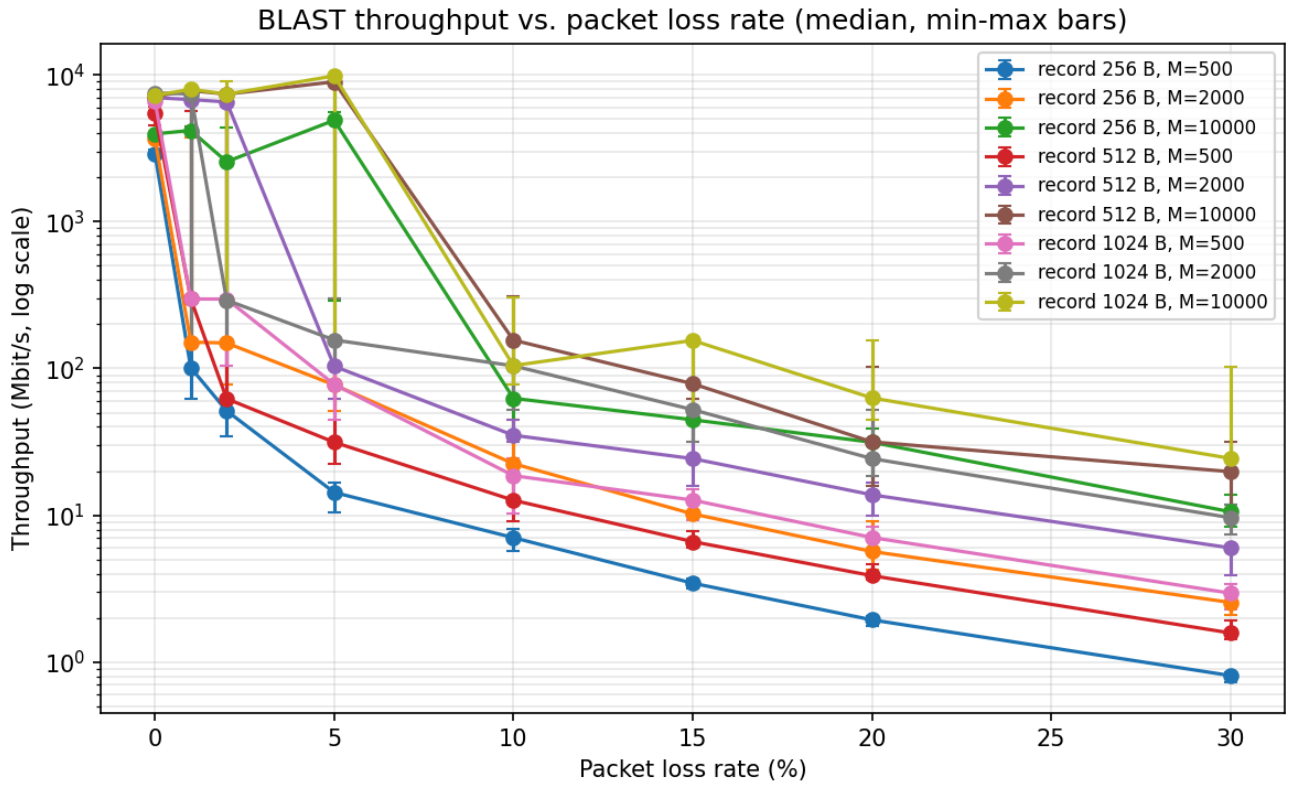


Figure 1. Median throughput of 5 runs for each record size and M; bars show the minimum and maximum. The y-axis is logarithmic.

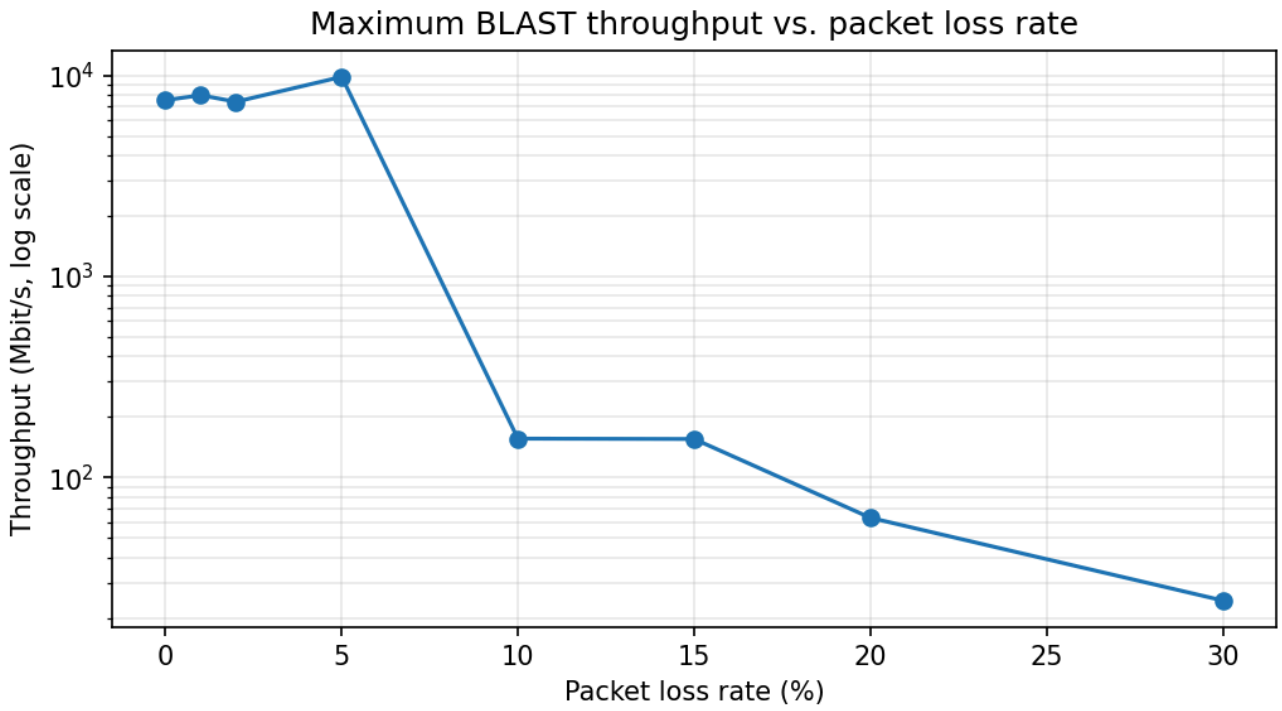


Figure 2. Maximum achievable throughput: the best setting at each loss rate (settings listed in Table 2).

Table 2. Maximum throughput (best median over all settings)

Loss rate	Max throughput (Mbit/s)	Best setting
0%	7523	1024 B, M = 2000
1%	7966	1024 B, M = 10000
2%	7387	1024 B, M = 10000
5%	9830	1024 B, M = 10000
10%	156	512 B, M = 10000
15%	155	1024 B, M = 10000
20%	63	1024 B, M = 10000
30%	24	1024 B, M = 10000

Table 3. Median throughput (Mbit/s) for every setting

Loss	256/500	256/2000	256/10000	512/500	512/2000	512/10000	1024/500	1024/2000	1024/10000
0%	2892	3689	3957	5485	6989	7132	6645	7523	7236
1%	101	151	4180	298	6782	7800	297	7403	7966
2%	51.5	149	2558	62.2	6533	7381	297	292	7388
5%	14.3	77.3	4900	31.4	104	8957	77.9	156	9830
10%	7.0	22.5	62.6	12.7	35.1	156	18.6	104	105
15%	3.5	10.2	44.7	6.6	24.3	78.7	12.7	52.4	155
20%	1.9	5.7	31.4	3.9	13.8	31.6	7.1	24.3	63.0
30%	0.8	2.5	10.6	1.6	6.0	19.8	3.0	9.6	24.4

(Column headings are record size in bytes / M.)

Table 4. Detail for the best setting (1024 B, M = 10000) and the smallest (256 B, M = 500)

Loss	1024/10000 median time (s)	timeouts	records resent	256/500 median time (s)	timeouts	records resent
0%	0.013	0.0	0	0.033	0.0	0
1%	0.012	0.0	109	0.950	3.2	403
2%	0.013	0.4	259	1.864	5.8	847
5%	0.010	0.4	604	6.702	22.6	2359
10%	0.917	2.6	1385	13.643	45.4	5211
15%	0.618	3.4	2101	27.809	92.8	8391
20%	1.524	4.8	2891	49.514	165.8	11637
30%	3.933	12.0	5066	118.473	397.0	19810

9. Discussion

Timeouts, not resent data, determine throughput. Without loss, the whole 12 MB file is sent in 10–35 ms (up to about 7.5 Gbit/s over loopback). Resending lost records costs very little: at 30% loss the best setting resends 5066 records, which takes only milliseconds. What costs time is losing a control packet. When an IS_BLAST_OVER or its REC_MISS is lost, the sender waits the full 300 ms timeout before asking again — more than 20 times the duration of an entire loss-free transfer. Transfer time is therefore approximately

time ≈ (time to send the data) + 0.3 s × (number of timeouts).

For 256 B / M = 500 at 30% loss, 397 timeouts × 0.3 s = 119 s, against a measured median of 118.5 s.

Throughput is bimodal. One timeout limits throughput to at most 96 Mbit ÷ 0.3 s ≈ 320 Mbit/s; two timeouts to about 160 Mbit/s; three to about 107 Mbit/s. The measured values cluster at exactly these levels (≈ 300, 150, 100 Mbit/s in Table 3), and a run with no timeout jumps back to several Gbit/s. This is why the min–max bars in Figure 1 are so wide at 1–5% loss, and why the report uses the median rather than the mean. The differences among runs with no timeout (e.g. 9830 Mbit/s at 5% versus 7523 Mbit/s at 0%) are measurement noise: a 10 ms transfer is sensitive to CPU scheduling, so those values should not be read as loss making the transfer faster.

How many timeouts to expect. One query round succeeds only if IS_BLAST_OVER and REC_MISS both get through, with probability $(1 - p)^2$. At 30% loss that is 0.49, so each round costs about one timeout on average. Each blast needs several rounds, since every round some resent records are lost again; for a blast of n packets, roughly $1 + \log(n) / \log(1/p)$ rounds. For 256 B / M = 500 (94 blasts of about 32 packets), this predicts about 3.9 rounds and 4.0 timeouts per blast; the measurement is 4.2 timeouts per blast.

Effect of M. A larger M means fewer blasts, so fewer IS_BLAST_OVER rounds and fewer chances to lose a control packet. M = 10000 was the best or nearly the best choice at every loss rate above 0%: at 30% loss, 1024 B records reach 24.4 Mbit/s with M = 10000 versus 3.0 Mbit/s with M = 500, an 8-fold difference. The cost of a large M is buffer memory: M × record size on each side (10 MB for M = 10000 and 1024-byte records).

Effect of record size. Larger records mean fewer packets for the same file, so fewer per-packet system calls and, under loss, fewer packets to lose and resend. Without loss, 1024 B and 512 B records reach 7–7.5 Gbit/s versus about 3.9 Gbit/s for 256 B records. Under loss, larger records also mean fewer packets per blast and therefore fewer rounds per blast.

Possible improvements. The results point to the timeout as the main cost. Useful changes would be an adaptive timeout based on the measured round-trip time (on loopback the RTT is well under 1 ms, so 300 ms is far too conservative), or sending IS_BLAST_OVER twice, since losing one copy alone would then no longer cause a timeout.

Limitations. Sender and receiver ran on one machine over loopback, so the absolute numbers reflect memory and CPU speed rather than a physical network; on Ethernet the loss-free rate would be bounded by the link speed (e.g. 1 Gbit/s). The relative behaviour — the cost of timeouts, and the benefit of larger M and records — does not depend on the link. On macOS, 1024-byte records travel 8 per packet instead of 16 because of the datagram size limit.

10. Conclusion

The implementation delivers all three phases of BLAST: connection setup with negotiation of the record size and M, blasts with selective retransmission through IS_BLAST_OVER and REC_MISS with multi-segment packets, and a disconnect with a linger period. Disk reading and writing run in separate threads from the network, so packets carry a variable number of records as the assignment requires. All 360 measured transfers were delivered correctly at loss rates up to 30%. Maximum throughput falls from about 7.5 Gbit/s without loss to 156 Mbit/s at 10% and 24 Mbit/s at 30%, and is governed by the number of 300 ms timeouts caused by lost control packets. The largest blast size (M = 10000) and the largest record size (1024 B) gave the highest throughput under loss.

11. Project role

This was an individual project. Md Mamun did all parts: protocol and packet design, the sender and receiver state machines, buffering and threads, the garbler, the experiments, and this report.

Appendix: source code

The source files `blast.h`, `garble.h`, `sender.c`, `receiver.c`, `run_experiments.sh`, and `plot.py` follow.

`blast.h`

```
// blast.h — packet formats shared by sender and receiver.
// All multi-byte fields travel in network byte order (htons/htonl).
#ifndef BLAST_H
#define BLAST_H
#include <stdint.h>

#define PKT_FILE_HDR       1
#define PKT_FILE_HDR_ACK   2
#define PKT_DATA           3
#define PKT_IS_BLAST_OVER  4
#define PKT_REC_MISS       5
#define PKT_DISCONNECT     6
#define PKT_DISCONNECT_ACK 7

#define MAX_SEGS           16
// Records per packet = min(16, MAX_DATA_BYTES / record_size).
// Linux allows UDP datagrams up to ~64 KB, so 16 x 1024-byte records fit.
// macOS caps datagrams at 9216 bytes by default, so there 1024-byte records -> 8 per packet.
#ifdef __APPLE__
#define MAX_DATA_BYTES     8192
#else
#define MAX_DATA_BYTES     (16 * 1024)
#endif
#define MAX_PKT            (2 + MAX_SEGS * 8 + MAX_DATA_BYTES)
#define MAX_NAME           256
#define MAX_RECS_PER_PKT   16
#define MIN_M              200
#define MAX_M              10000

struct __attribute__((packed)) file_hdr {
    uint8_t  type;                 // PKT_FILE_HDR
    uint32_t file_size;            // bytes
    uint16_t record_size;          // 256, 512 or 1024
    uint32_t blast_size;           // M, records per blast (proposed)
    char     filename[MAX_NAME];   // name the receiver stores the file under
};

struct __attribute__((packed)) file_hdr_ack {
    uint8_t  type;                 // PKT_FILE_HDR_ACK
    uint8_t  status;               // 0 = OK, 1 = rejected
    uint16_t record_size;          // agreed
    uint32_t blast_size;           // agreed (receiver may lower it)
};

// DATA packet = data_hdr, then nseg x seg (the segment list), then record bytes
// for segment 1, segment 2, ... in order. Records are numbered from 1.
struct __attribute__((packed)) data_hdr {
    uint8_t type;                  // PKT_DATA
    uint8_t nseg;                  // N, number of segments (1..MAX_SEGS)
};
struct __attribute__((packed)) seg {
    uint32_t st_rec;               // first record of segment
    uint32_t end_rec;              // last record of segment (inclusive)
};

// ---- reliability ----
#define MAX_MISS_RANGES    512   // max ranges reported in one REC_MISS

struct __attribute__((packed)) is_blast_over {
    uint8_t  type;                 // PKT_IS_BLAST_OVER
    uint32_t seq;                  // query number; REC_MISS echoes it so stale replies are ignored
    uint32_t mst;                  // first record of the blast
    uint32_t mfin;                 // last record of the blast
};
struct __attribute__((packed)) rec_miss {
    uint8_t  type;                 // PKT_REC_MISS
    uint32_t seq;                  // echo of is_blast_over.seq
    uint32_t mst, mfin;            // echo of the blast range
    uint16_t nranges;              // 0 = nothing missing. Followed by nranges x struct seg.
};

// ---- disconnect ----
#define LINGER_SEC         3     // receiver keeps answering this long after DISCONNECT

struct __attribute__((packed)) ctl {      // DISCONNECT and DISCONNECT_ACK carry only a type
    uint8_t type;
};

#endif
```

garble.h

```
// garble.h — minimal "garbler": drops outgoing packets at a given rate.
// Both sender and receiver send through g_sendto(), so loss hits data AND control packets.
#ifndef GARBLE_H
#define GARBLE_H
#include <stdlib.h>
#include <sys/socket.h>

static double g_loss = 0.0;        // probability of dropping each outgoing packet
static long g_attempted = 0, g_dropped = 0;

static inline void garble_init(double rate, unsigned seed) {
    g_loss = rate;
    srand48(seed);
}

static inline ssize_t g_sendto(int sock, const void *buf, size_t len, int flags,
                               const struct sockaddr *to, socklen_t tolen) {
    g_attempted++;
    if (g_loss > 0 && drand48() < g_loss) { g_dropped++; return (ssize_t)len; } // "sent" but lost
    return sendto(sock, buf, len, flags, to, tolen);
}
#endif
```

sender.c

```
// BLAST sender. Usage: ./sender <ip> <port> <file> <record_size> <M> [loss_rate] [dest_name]
// dest_name is the name the receiver stores the file under (default: the file's basename).
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <unistd.h>
#include <time.h>
#include <errno.h>
#include <libgen.h>
#include <pthread.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <sys/stat.h>
#include <sys/time.h>
#include "blast.h"
#include "garble.h"

#define HDR_TIMEOUT_MS    1000
#define HDR_MAX_TRIES     20
#define BLAST_TIMEOUT_MS  300     // wait for REC_MISS this long before re-asking
#define QUERY_MAX_TRIES   30
#define READ_CHUNK_MAX    40      // each disk read fetches 1..READ_CHUNK_MAX records (random)

// ---- disk reader thread: file -> buffer B' ----
// B' is a ring of M record slots; record r lives in slot (r-1) % M. The reader runs
// independently of the network: it reads chunks of random size and publishes how far
// it got (avail). It may fill record r only once r <= released + M, i.e. after the
// blast that used that slot has been confirmed by an empty REC_MISS.
static struct {
    pthread_mutex_t mu; pthread_cond_t cv;
    FILE *f;
    uint8_t *ring; long cap;
    int rec_size;
    long total_recs;
    long avail;                    // records 1..avail are in B'
    long released;                 // records 1..released are confirmed; their slots can be reused
    long reads;                    // number of disk reads
} R = { .mu = PTHREAD_MUTEX_INITIALIZER, .cv = PTHREAD_COND_INITIALIZER };

static void *reader(void *arg) {
    (void)arg;
    unsigned seed = (unsigned)time(NULL) ^ (unsigned)getpid();
    long next = 1;
    while (next <= R.total_recs) {
        pthread_mutex_lock(&R.mu);
        while (next > R.released + R.cap) pthread_cond_wait(&R.cv, &R.mu);
        long room = R.released + R.cap - next + 1;
        pthread_mutex_unlock(&R.mu);

        long want = 1 + rand_r(&seed) % READ_CHUNK_MAX;
        if (want > room) want = room;
        if (want > R.total_recs - next + 1) want = R.total_recs - next + 1;
        long slot = (next - 1) % R.cap;
        if (want > R.cap - slot) want = R.cap - slot;          // a read never wraps the ring
        size_t got = fread(R.ring + slot * R.rec_size, 1, (size_t)want * R.rec_size, R.f);
        if (got == 0) { fprintf(stderr, "disk read failed\n"); exit(1); }
        long n = ((long)got + R.rec_size - 1) / R.rec_size;   // only the file's last record is short

        pthread_mutex_lock(&R.mu);
        next += n;
        R.avail = next - 1;
        R.reads++;
        pthread_cond_broadcast(&R.cv);
        pthread_mutex_unlock(&R.mu);
```

```
    }
    return NULL;
}

// block until record r has been read from disk; returns how many records are available
static long wait_avail(long r) {
    pthread_mutex_lock(&R.mu);
    while (R.avail < r) pthread_cond_wait(&R.cv, &R.mu);
    long a = R.avail;
    pthread_mutex_unlock(&R.mu);
    return a;
}

static void release_upto(long r) {
    pthread_mutex_lock(&R.mu);
    R.released = r;
    pthread_cond_broadcast(&R.cv);
    pthread_mutex_unlock(&R.mu);
}

// ---- packet builder: packs records (possibly non-contiguous) into DATA packets ----
struct hseg { uint32_t st, end; };
struct builder {
    int sock; struct sockaddr_in *dst;
    int max_recs;
    struct hseg segs[MAX_SEGS]; int nseg, nrec;
    uint8_t data[MAX_DATA_BYTES]; int dlen;
    long pkts_sent, recs_sent, partial_pkts;   // partial = fewer than max_recs records
};

static void flush_pkt(struct builder *b) {
    if (b->nrec == 0) return;
    uint8_t pkt[MAX_PKT];
    int off = 0;
    pkt[off++] = PKT_DATA;
    pkt[off++] = (uint8_t)b->nseg;
    for (int i = 0; i < b->nseg; i++) {
        struct seg s = { htonl(b->segs[i].st), htonl(b->segs[i].end) };
        memcpy(pkt + off, &s, sizeof s); off += sizeof s;
    }
    memcpy(pkt + off, b->data, b->dlen); off += b->dlen;
    for (;;) {
        ssize_t r = g_sendto(b->sock, pkt, off, 0, (struct sockaddr *)b->dst, sizeof *b->dst);
        if (r >= 0) break;
        if (errno == ENOBUFS || errno == EAGAIN) { usleep(200); continue; }
        perror("sendto"); exit(1);
    }
    b->pkts_sent++;
    b->recs_sent += b->nrec;
    if (b->nrec < b->max_recs) b->partial_pkts++;
    b->nseg = b->nrec = b->dlen = 0;
}

static void add_record(struct builder *b, uint32_t rec, const uint8_t *p, int len) {
    if (b->nrec == b->max_recs) flush_pkt(b);
    if (b->nseg > 0 && b->segs[b->nseg - 1].end + 1 == rec) {
        b->segs[b->nseg - 1].end = rec;
    } else {
        if (b->nseg == MAX_SEGS) flush_pkt(b);
        b->segs[b->nseg].st = b->segs[b->nseg].end = rec;
        b->nseg++;
    }
    memcpy(b->data + b->dlen, p, len);
    b->dlen += len; b->nrec++;
}

static off_t g_file_size;

static const uint8_t *rec_ptr(long r) { return R.ring + ((r - 1) % R.cap) * R.rec_size; }
static int rec_len(long r) {
    return r == R.total_recs ? (int)(g_file_size - (off_t)(r - 1) * R.rec_size) : R.rec_size;
}

static void set_timeout(int sock, int ms) {
    struct timeval tv = { ms / 1000, (ms % 1000) * 1000 };
    setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);
}

int main(int argc, char **argv) {
    if (argc < 6 || argc > 8) {
        fprintf(stderr, "usage: %s <ip> <port> <file> <record_size> <M> [loss_rate] [dest_name]\n", argv[0]);
        return 1;
    }
    const char *path = argv[3];
    int rec_size = atoi(argv[4]);
    int M = atoi(argv[5]);
    garble_init(argc >= 7 ? atof(argv[6]) : 0.0, (unsigned)time(NULL) ^ getpid());
    if (rec_size != 256 && rec_size != 512 && rec_size != 1024) {
        fprintf(stderr, "record_size must be 256, 512 or 1024\n"); return 1;
    }
    if (M < MIN_M || M > MAX_M) { fprintf(stderr, "M must be in %d..%d\n", MIN_M, MAX_M); return 1; }
    struct stat st;
    if (stat(path, &st) < 0 || st.st_size == 0) { fprintf(stderr, "cannot read file (or empty)\n"); return 1; }
    g_file_size = st.st_size;

    int sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0) { perror("socket"); return 1; }
```

```
    set_timeout(sock, HDR_TIMEOUT_MS);

    struct sockaddr_in dst = {0};
    dst.sin_family = AF_INET;
    dst.sin_port = htons(atoi(argv[2]));
    if (inet_pton(AF_INET, argv[1], &dst.sin_addr) != 1) { fprintf(stderr, "bad ip\n"); return 1; }

    // ---- Phase 1: connection setup ----
    struct file_hdr hdr = {0};
    hdr.type = PKT_FILE_HDR;
    hdr.file_size = htonl((uint32_t)st.st_size);
    hdr.record_size = htons((uint16_t)rec_size);
    hdr.blast_size = htonl((uint32_t)M);
    if (argc == 8) {
        strncpy(hdr.filename, argv[7], MAX_NAME - 1);
    } else {
        char *tmp = strdup(path);
        strncpy(hdr.filename, basename(tmp), MAX_NAME - 1);
        free(tmp);
    }

    int connected = 0;
    struct file_hdr_ack ack;
    for (int attempt = 1; attempt <= HDR_MAX_TRIES && !connected; attempt++) {
        g_sendto(sock, &hdr, sizeof hdr, 0, (struct sockaddr *)&dst, sizeof dst);
        for (;;) {
            ssize_t n = recvfrom(sock, &ack, sizeof ack, 0, NULL, NULL);
            if (n < 0) {
                if (errno == EAGAIN || errno == EWOULDBLOCK) break;
                perror("recvfrom"); return 1;
            }
            if (n == sizeof ack && ack.type == PKT_FILE_HDR_ACK) { connected = 1; break; }
        }
    }
    if (!connected) { fprintf(stderr, "no FILE_HDR_ACK, giving up\n"); return 1; }
    if (ack.status != 0) { fprintf(stderr, "receiver rejected the transfer\n"); return 1; }

    rec_size = ntohs(ack.record_size);
    M = (int)ntohl(ack.blast_size);
    long total_recs = (st.st_size + rec_size - 1) / rec_size;
    printf("connected: record_size=%d, M=%d, file=%lld bytes -> '%s', %ld records, loss=%.0f%%\n",
           rec_size, M, (long long)st.st_size, hdr.filename, total_recs, g_loss * 100);

    // ---- Phase 2: blasts ----
    R.f = fopen(path, "rb");
    if (!R.f) { perror("fopen"); return 1; }
    R.rec_size = rec_size;
    R.cap = M;
    R.total_recs = total_recs;
    R.ring = malloc((size_t)M * rec_size);
    if (!R.ring) { perror("malloc"); return 1; }

    struct builder b = {0};
    b.sock = sock; b.dst = &dst;
    b.max_recs = MAX_DATA_BYTES / rec_size < MAX_RECS_PER_PKT ? MAX_DATA_BYTES / rec_size
                                                              : MAX_RECS_PER_PKT;
    set_timeout(sock, BLAST_TIMEOUT_MS);

    long resent_recs = 0, queries = 0, timeouts = 0, miss_rcvd = 0;
    uint32_t seq = 0;
    uint8_t rbuf[MAX_PKT];
    struct timeval t0, t1;
    gettimeofday(&t0, NULL);

    pthread_t rd;
    if (pthread_create(&rd, NULL, reader, NULL) != 0) { fprintf(stderr, "pthread_create failed\n"); return 1; }

    for (long first = 1; first <= total_recs; first += M) {
        long last = first + M - 1 < total_recs ? first + M - 1 : total_recs;

        // Send records as soon as the disk reader makes them available. Whatever is
        // ready goes out right away, so a packet carries anywhere from 1 to max_recs records.
        long r = first;
        while (r <= last) {
            long a = wait_avail(r);
            if (a > last) a = last;
            for (; r <= a; r++) add_record(&b, (uint32_t)r, rec_ptr(r), rec_len(r));
            flush_pkt(&b);
        }

        // Ask "is the blast over?" until the receiver reports nothing missing.
        int rounds = 0;
        for (;;) {
            seq++;
            struct is_blast_over q = { PKT_IS_BLAST_OVER, htonl(seq), htonl((uint32_t)first), htonl((uint32_t)last) };
            struct rec_miss *rm = NULL;
            ssize_t n = 0;
            for (int tries = 0; tries < QUERY_MAX_TRIES && !rm; tries++) {
                g_sendto(sock, &q, sizeof q, 0, (struct sockaddr *)&dst, sizeof dst);
                queries++;
                for (;;) {
                    n = recvfrom(sock, rbuf, sizeof rbuf, 0, NULL, NULL);
                    if (n < 0) {
                        if (errno == EAGAIN || errno == EWOULDBLOCK) { timeouts++; break; }
                        perror("recvfrom"); return 1;
                    }
                    struct rec_miss *c = (struct rec_miss *)rbuf;
```

```
                    if (n >= (ssize_t)sizeof *c && c->type == PKT_REC_MISS && ntohl(c->seq) == seq) { rm = c; break; }
                }
            }
            if (!rm) { fprintf(stderr, "receiver not responding, giving up\n"); return 1; }
            miss_rcvd++;
            int nr = ntohs(rm->nranges);
            if (n < (ssize_t)(sizeof *rm + nr * sizeof(struct seg))) continue;   // malformed: re-ask
            if (nr == 0) break;                                                  // blast complete
            rounds++;
            for (int i = 0; i < nr; i++) {
                struct seg s;
                memcpy(&s, rbuf + sizeof *rm + i * sizeof s, sizeof s);
                long st_r = ntohl(s.st_rec), en_r = ntohl(s.end_rec);
                if (st_r < first) st_r = first;
                if (en_r > last) en_r = last;
                for (long rr = st_r; rr <= en_r; rr++) {
                    add_record(&b, (uint32_t)rr, rec_ptr(rr), rec_len(rr));
                    resent_recs++;
                }
            }
            flush_pkt(&b);
        }
        release_upto(last);            // B' slots of this blast may now be refilled from disk
        printf("blast %ld-%ld done (%d retransmit rounds)\n", first, last, rounds);
    }
    gettimeofday(&t1, NULL);
    pthread_join(rd, NULL);
    fclose(R.f);
    double secs = (t1.tv_sec - t0.tv_sec) + (t1.tv_usec - t0.tv_usec) / 1e6;

    // ---- Phase 3: disconnect ----
    // Every record is already confirmed, so the transfer succeeded even if the
    // DISCONNECT_ACK never arrives; we just retry a few times and then leave.
    struct ctl bye = { PKT_DISCONNECT };
    int closed = 0, bye_sent = 0;
    for (int tries = 0; tries < 10 && !closed; tries++) {
        g_sendto(sock, &bye, sizeof bye, 0, (struct sockaddr *)&dst, sizeof dst);
        bye_sent++;
        for (;;) {
            ssize_t n = recvfrom(sock, rbuf, sizeof rbuf, 0, NULL, NULL);
            if (n < 0) {
                if (errno == EAGAIN || errno == EWOULDBLOCK) break;
                perror("recvfrom"); return 1;
            }
            if (n == sizeof(struct ctl) && rbuf[0] == PKT_DISCONNECT_ACK) { closed = 1; break; }
        }
    }
    if (closed) printf("disconnected cleanly (%d DISCONNECT sent)\n", bye_sent);
    else printf("no DISCONNECT_ACK after %d tries; all data was already confirmed, exiting\n", bye_sent);
    printf("---- stats ----\n");
    printf("data packets sent: %ld, records resent: %ld\n", b.pkts_sent, resent_recs);
    printf("records per packet: avg %.2f (max %d), partial packets: %ld, disk reads: %ld\n",
           (double)b.recs_sent / b.pkts_sent, b.max_recs, b.partial_pkts, R.reads);
    printf("IS_BLAST_OVER sent: %ld, REC_MISS received: %ld, timeouts: %ld\n", queries, miss_rcvd, timeouts);
    printf("packets dropped by garbler (sender side): %ld of %ld\n", g_dropped, g_attempted);
    printf("time %.3f s, throughput %.2f Mbit/s\n", secs, st.st_size * 8.0 / secs / 1e6);
    return 0;
}
```

receiver.c

```
// BLAST receiver. Usage: ./receiver <port> <max_M> [loss_rate]
// Runs forever and serves one transfer after another. Each file is stored under the
// name given in FILE_HDR (written as <name>.part, renamed to <name> once complete).
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <time.h>
#include <libgen.h>
#include <pthread.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <sys/time.h>
#include "blast.h"
#include "garble.h"

enum state { IDLE, ACTIVE, LINGER };

// everything we know about the current transfer
static struct {
    enum state st;
    int fd;
    uint32_t file_size, M, total_recs, nrecv;
    uint16_t rec_size;
    uint8_t *got;                  // got[r] = 1 once record r has arrived
    long pkts, dup_recs, miss_sent;
    double linger_until;
    char name[MAX_NAME], part[MAX_NAME + 8];
} S = { .st = IDLE, .fd = -1 };

// ---- buffer B' and disk writer thread ----
// B' is a ring of M record slots; record r lives in slot (r-1) % M. The network side
```

```
// copies records into B' and advances contig (records 1..contig have all arrived).
// The writer thread independently writes records written+1..contig to disk. The network
// side may reuse the slot of record r only once r <= written + M.
static struct {
    pthread_mutex_t mu; pthread_cond_t cv;
    pthread_t tid;
    uint8_t *ring;
    uint32_t contig, written;
    int stop;
    long writes;                    // number of disk writes
} W = { .mu = PTHREAD_MUTEX_INITIALIZER, .cv = PTHREAD_COND_INITIALIZER };

static void *writer(void *arg) {
    (void)arg;
    pthread_mutex_lock(&W.mu);
    for (;;) {
        while (W.written == W.contig && !W.stop) pthread_cond_wait(&W.cv, &W.mu);
        if (W.written == W.contig) break;                   // stop requested and all flushed
        uint32_t r = W.written + 1, target = W.contig;
        pthread_mutex_unlock(&W.mu);

        uint32_t slot = (r - 1) % S.M;
        uint32_t n = target - r + 1;
        if (n > S.M - slot) n = S.M - slot;                 // a write never wraps the ring
        off_t off = (off_t)(r - 1) * S.rec_size;
        size_t len = (size_t)n * S.rec_size;
        if (off + (off_t)len > (off_t)S.file_size) len = S.file_size - off;
        const uint8_t *p = W.ring + (size_t)slot * S.rec_size;
        while (len > 0) {
            ssize_t k = pwrite(S.fd, p, len, off);
            if (k < 0) { perror("pwrite"); exit(1); }
            p += k; off += k; len -= k;
        }

        pthread_mutex_lock(&W.mu);
        W.written += n;
        W.writes++;
        pthread_cond_broadcast(&W.cv);
    }
    pthread_mutex_unlock(&W.mu);
    return NULL;
}

// stop the writer after it has flushed everything that is contiguous
static void writer_finish(void) {
    pthread_mutex_lock(&W.mu);
    W.stop = 1;
    pthread_cond_broadcast(&W.cv);
    pthread_mutex_unlock(&W.mu);
    pthread_join(W.tid, NULL);
    free(W.ring); W.ring = NULL;
}

static double now_sec(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

static void end_session(void) {
    if (S.st == ACTIVE) writer_finish();               // transfer abandoned mid-way
    if (S.fd >= 0) close(S.fd);
    S.fd = -1;
    free(S.got); S.got = NULL;
    S.st = IDLE;
}

int main(int argc, char **argv) {
    if (argc < 3 || argc > 4) { fprintf(stderr, "usage: %s <port> <max_M> [loss_rate]\n", argv[0]); return 1; }
    int max_M = atoi(argv[2]);
    if (max_M < MIN_M || max_M > MAX_M) { fprintf(stderr, "max_M must be in %d..%d\n", MIN_M, MAX_M); return 1; }
    garble_init(argc == 4 ? atof(argv[3]) : 0.0, (unsigned)time(NULL) ^ getpid());
    setvbuf(stdout, NULL, _IOLBF, 0);

    int sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0) { perror("socket"); return 1; }
    int rcvbuf = 4 * 1024 * 1024;
    setsockopt(sock, SOL_SOCKET, SO_RCVBUF, &rcvbuf, sizeof rcvbuf);
    struct timeval tv = {0, 200000};                    // wake up every 200 ms to check the linger timer
    setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(atoi(argv[1]));
    if (bind(sock, (struct sockaddr *)&addr, sizeof addr) < 0) { perror("bind"); return 1; }
    printf("receiver listening on UDP port %s (max M = %d, loss=%.0f%%)\n", argv[1], max_M, g_loss * 100);

    uint8_t buf[MAX_PKT];
    for (;;) {
        // linger over? -> forget the old transfer, get ready for a new one
        if (S.st == LINGER && now_sec() >= S.linger_until) {
            end_session();
            printf("linger over, ready for next transfer\n");
        }

        struct sockaddr_in from;
        socklen_t flen = sizeof from;
```

```
        ssize_t n = recvfrom(sock, buf, sizeof buf, 0, (struct sockaddr *)&from, &flen);
        if (n < 1) continue;                            // timeout or error: loop and re-check timer

        switch (buf[0]) {
        case PKT_FILE_HDR: {
            if (n != sizeof(struct file_hdr)) break;
            struct file_hdr *h = (struct file_hdr *)buf;
            struct file_hdr_ack ack = {0};
            ack.type = PKT_FILE_HDR_ACK;
            uint16_t rs = ntohs(h->record_size);
            uint32_t m = ntohl(h->blast_size);
            if (rs != 256 && rs != 512 && rs != 1024) {
                ack.status = 1;
            } else if (S.st == ACTIVE) {
                printf("duplicate FILE_HDR, re-sending ACK\n");     // our earlier ACK was lost
            } else {
                if (S.st == LINGER) end_session();                  // previous transfer is finished
                h->filename[MAX_NAME - 1] = '\0';
                snprintf(S.name, sizeof S.name, "%s", basename(h->filename));  // no directories
                snprintf(S.part, sizeof S.part, "%s.part", S.name);
                S.fd = open(S.part, O_CREAT | O_WRONLY | O_TRUNC, 0644);
                if (S.fd < 0) { perror("open"); ack.status = 1; }
                else {
                    S.rec_size = rs;
                    S.file_size = ntohl(h->file_size);
                    S.M = m > (uint32_t)max_M ? (uint32_t)max_M : m;
                    S.total_recs = (S.file_size + S.rec_size - 1) / S.rec_size;
                    S.got = calloc(S.total_recs + 1, 1);
                    S.nrecv = 0; S.pkts = 0; S.dup_recs = 0; S.miss_sent = 0;
                    g_dropped = 0; g_attempted = 0;                 // per-transfer garbler stats
                    W.ring = malloc((size_t)S.M * S.rec_size);
                    W.contig = W.written = 0; W.stop = 0; W.writes = 0;
                    if (pthread_create(&W.tid, NULL, writer, NULL) != 0) {
                        fprintf(stderr, "pthread_create failed\n"); return 1;
                    }
                    S.st = ACTIVE;
                    printf("FILE_HDR: '%s' %u bytes, record_size=%u, M=%u, %u records\n",
                           S.name, S.file_size, S.rec_size, S.M, S.total_recs);
                }
            }
            if (ack.status == 0) { ack.record_size = htons(S.rec_size); ack.blast_size = htonl(S.M); }
            g_sendto(sock, &ack, sizeof ack, 0, (struct sockaddr *)&from, flen);
            break;
        }
        case PKT_DATA: {
            if (S.st != ACTIVE || n < (ssize_t)sizeof(struct data_hdr)) break;
            S.pkts++;
            int nseg = buf[1];
            size_t off = sizeof(struct data_hdr) + (size_t)nseg * sizeof(struct seg);
            if (nseg < 1 || nseg > MAX_SEGS || (size_t)n < off) break;
            for (int i = 0; i < nseg; i++) {
                struct seg s;
                memcpy(&s, buf + sizeof(struct data_hdr) + i * sizeof s, sizeof s);
                uint32_t st = ntohl(s.st_rec), end = ntohl(s.end_rec);
                if (st < 1 || end < st || end > S.total_recs) break;
                uint64_t b_end = (uint64_t)end * S.rec_size;
                if (b_end > S.file_size) b_end = S.file_size;
                if (off + (b_end - (uint64_t)(st - 1) * S.rec_size) > (size_t)n) break;
                for (uint32_t r = st; r <= end; r++) {
                    size_t len = r == S.total_recs ? S.file_size - (size_t)(r - 1) * S.rec_size : S.rec_size;
                    if (S.got[r]) { S.dup_recs++; off += len; continue; }
                    // wait until the writer has freed this record's slot in B'
                    pthread_mutex_lock(&W.mu);
                    while (r > W.written + S.M) pthread_cond_wait(&W.cv, &W.mu);
                    pthread_mutex_unlock(&W.mu);
                    memcpy(W.ring + (size_t)((r - 1) % S.M) * S.rec_size, buf + off, len);
                    off += len;
                    S.got[r] = 1; S.nrecv++;
                }
            }
            // hand newly contiguous records to the writer
            uint32_t c = W.contig;
            while (c < S.total_recs && S.got[c + 1]) c++;
            if (c != W.contig) {
                pthread_mutex_lock(&W.mu);
                W.contig = c;
                pthread_cond_broadcast(&W.cv);
                pthread_mutex_unlock(&W.mu);
            }
            break;
        }
        case PKT_IS_BLAST_OVER: {
            // Also answered while lingering, so a retransmitted query still gets a REC_MISS.
            if (S.st == IDLE || n != sizeof(struct is_blast_over)) break;
            struct is_blast_over *q = (struct is_blast_over *)buf;
            uint32_t mst = ntohl(q->mst), mfin = ntohl(q->mfin);
            if (mst < 1 || mst > mfin || mfin > S.total_recs) break;

            uint8_t out[sizeof(struct rec_miss) + MAX_MISS_RANGES * sizeof(struct seg)];
            struct rec_miss *rm = (struct rec_miss *)out;
            int nr = 0;
            uint32_t r = mst;
            while (r <= mfin && nr < MAX_MISS_RANGES) {
                if (S.got[r]) { r++; continue; }
                uint32_t st = r;
                while (r <= mfin && !S.got[r]) r++;
                struct seg s = { htonl(st), htonl(r - 1) };
```

```
                memcpy(out + sizeof *rm + nr * sizeof s, &s, sizeof s);
                nr++;
            }
            rm->type = PKT_REC_MISS;
            rm->seq = q->seq;
            rm->mst = q->mst; rm->mfin = q->mfin;
            rm->nranges = htons((uint16_t)nr);
            g_sendto(sock, out, sizeof *rm + nr * sizeof(struct seg), 0, (struct sockaddr *)&from, flen);
            S.miss_sent++;
            break;
        }
        case PKT_DISCONNECT: {
            if (S.st == IDLE) break;
            if (S.st == ACTIVE) {
                writer_finish();                          // flush B' to disk
                if (S.fd >= 0) { close(S.fd); S.fd = -1; }   // keep got[] for the linger period
                printf("DISCONNECT: %u/%u records, %ld data packets received, %ld duplicate records, "
                       "%ld disk writes, %ld REC_MISS sent, receiver-side garbler dropped %ld of %ld\n",
                       S.nrecv, S.total_recs, S.pkts, S.dup_recs, W.writes, S.miss_sent, g_dropped, g_attempted);
                if (S.nrecv != S.total_recs) printf("WARNING: file incomplete, left as '%s'\n", S.part);
                else if (rename(S.part, S.name) < 0) perror("rename");
                S.st = LINGER;
                S.linger_until = now_sec() + LINGER_SEC;
            }
            struct ctl ackp = { PKT_DISCONNECT_ACK };        // re-ACK if sender retransmits DISCONNECT
            g_sendto(sock, &ackp, sizeof ackp, 0, (struct sockaddr *)&from, flen);
            break;
        }
        default:
            printf("ignoring packet type %d\n", buf[0]);
        }
    }
}
```

Makefile

```
CC ?= cc
CFLAGS ?= -O2 -Wall -Wextra

all: sender receiver

sender: sender.c blast.h garble.h
        $(CC) $(CFLAGS) -pthread -o $@ sender.c

receiver: receiver.c blast.h garble.h
        $(CC) $(CFLAGS) -pthread -o $@ receiver.c

clean:
        rm -f sender receiver

.PHONY: all clean
```

run_experiments.sh

```
#!/bin/bash
# Run BLAST at several packet-loss rates, record sizes and blast sizes (M);
# collect statistics into results.csv.
# Usage: ./run_experiments.sh <file> "<record sizes>" "<M values>" <runs_per_setting> [loss rates...]
# Example: ./run_experiments.sh test12mb.bin "256 512 1024" "500 2000 10000" 5
#          ./run_experiments.sh test12mb.bin 512 1000 3 0 0.05 0.1
#
# Two machines (the setup the project asks for): build the receiver on the other host,
# make sure "ssh <host>" works without a password, then set
#   RX_SSH=user@10.0.0.2  RX_IP=10.0.0.2  RX_DIR=Blast   ./run_experiments.sh ...
# RX_DIR is the directory (on the receiver host) that holds ./receiver.
# Locally, the receiver runs in RX_DIR (default: a temp directory). Keep it outside folders that
# iCloud/Dropbox sync: they may rename files mid-transfer and break the final rename of <name>.part.

FILE=${1:?usage: $0 <file> "<record sizes>" "<M values>" <runs> [rates...]}
RSS=${2:?record sizes}
MS=${3:?M values}
RUNS=${4:?runs per setting}
RATES=("${@:5}")
[ ${#RATES[@]} -eq 0 ] && RATES=(0 0.01 0.02 0.05 0.10 0.15 0.20 0.30)

PORT=9500
OUT=${OUT:-results.csv}
NAME=$(basename "$FILE")
DEST=recv_$NAME
RX_IP=${RX_IP:-127.0.0.1}
if [ -n "$RX_SSH" ]; then
    RX_DIR=${RX_DIR:-.}; RECEIVER=./receiver
else
    RX_DIR=${RX_DIR:-${TMPDIR:-/tmp}/blast_rx}; RECEIVER="$PWD/receiver"
    mkdir -p "$RX_DIR"
fi

# run a shell command in RX_DIR where the receiver lives (locally, or over ssh)
rx() { if [ -n "$RX_SSH" ]; then ssh "$RX_SSH" "cd $RX_DIR && $1"; else (cd "$RX_DIR" && bash -c "$1"); fi; }

SUM=$(cksum < "$FILE")
```

```
echo
"loss,record_size,M,run,ok,time_s,throughput_mbps,data_pkts,records_resent,avg_recs_per_pkt,partial_pkts,timeouts,tx_dropped,tx_total,rx_dropped,rx_total"
> $OUT

for loss in "${RATES[@]}"; do
    RPID=$(rx "nohup $RECEIVER $PORT 10000 $loss > rx_$loss.log 2>&1 & echo \$!")
    sleep 0.5
    for rs in $RSS; do
    for M in $MS; do
    for run in $(seq 1 $RUNS); do
        rx "rm -f $DEST"
        out=$(./sender $RX_IP $PORT "$FILE" $rs $M $loss $DEST 2>&1)
        sleep 0.3
        [ "$(rx "cksum < $DEST 2>/dev/null")" = "$SUM" ] && ok=1 || ok=0
        if [ $ok = 0 ]; then                    # keep evidence of a failed transfer
            mkdir -p failures
            f=failures/loss${loss}_rs${rs}_M${M}_run${run}
            echo "$out" > $f.sender.log
            rx "ls -la $DEST* 2>&1; cksum < $DEST 2>&1" > $f.receiver.txt
        fi

        tm=$(echo "$out" | sed -n 's/^time \([0-9.]*\) s.*/\1/p')
        tp=$(echo "$out" | sed -n 's/.*throughput \([0-9.]*\) Mbit.*/\1/p')
        pk=$(echo "$out" | sed -n 's/^data packets sent: \([0-9]*\),.*/\1/p')
        rr=$(echo "$out" | sed -n 's/.*records resent: \([0-9]*\).*/\1/p')
        av=$(echo "$out" | sed -n 's/^records per packet: avg \([0-9.]*\).*/\1/p')
        pp=$(echo "$out" | sed -n 's/.*partial packets: \([0-9]*\).*/\1/p')
        to=$(echo "$out" | sed -n 's/.*timeouts: \([0-9]*\).*/\1/p')
        tx=$(echo "$out" | sed -n 's/.*sender side): \([0-9]*\) of \([0-9]*\).*/\1,\2/p')
        rxs=$(rx "grep 'receiver-side garbler dropped' rx_$loss.log | tail -1" \
             | sed -n 's/.*dropped \([0-9]*\) of \([0-9]*\).*/\1,\2/p')

        echo "$loss,$rs,$M,$run,$ok,$tm,$tp,$pk,$rr,$av,$pp,$to,${tx:-0,0},${rxs:-0,0}" >> $OUT
        echo "loss=$loss rs=$rs M=$M run=$run ok=$ok time=${tm}s throughput=${tp} Mbit/s"
    done
    done
    done
    rx "kill $RPID" 2>/dev/null
    rx "cat rx_$loss.log" > rx_$loss.log          # keep a copy of the receiver log here
    sleep 0.3
done
rx "rm -f $DEST"
echo "done -> $OUT"
```

plot.py

```
# Plot throughput vs. packet-loss rate from results.csv.
# Produces throughput_vs_loss.png (one line per record size / M setting) and
# throughput_max.png (best setting at each loss rate = maximum achievable throughput).
# Lines show the median of the runs; error bars show min..max. The median is used because
# throughput is bimodal under loss: a run either has no timeout or loses >= 300 ms to one.
# Usage: python3 plot.py [results.csv]    (needs: pip3 install matplotlib)
import csv, sys, statistics as st
from collections import defaultdict
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

path = sys.argv[1] if len(sys.argv) > 1 else "results.csv"
rows = defaultdict(lambda: defaultdict(list))        # (record_size, M) -> loss -> [rows]
for r in csv.DictReader(open(path)):
    if r["ok"] == "1" and r["throughput_mbps"]:
        rows[(int(r["record_size"]), int(r["M"]))][float(r["loss"])].append(r)

def tput(rs): return [float(r["throughput_mbps"]) for r in rs]

print(f"{'rec':>5} {'M':>6} {'loss%':>6} {'runs':>4} {'median Mbit/s':>14} {'min':>9} {'max':>9} "
      f"{'resent recs':>12} {'timeouts':>9}")
plt.figure(figsize=(8, 5))
for (rs, M), by_loss in sorted(rows.items()):
    losses = sorted(by_loss)
    med = [st.median(tput(by_loss[l])) for l in losses]
    lo = [min(tput(by_loss[l])) for l in losses]
    hi = [max(tput(by_loss[l])) for l in losses]
    for l, m, a, b in zip(losses, med, lo, hi):
        resent = st.mean(float(r["records_resent"]) for r in by_loss[l])
        to = st.mean(float(r["timeouts"]) for r in by_loss[l])
        print(f"{rs:5d} {M:6d} {l*100:6.1f} {len(by_loss[l]):4d} {m:14.2f} {a:9.2f} {b:9.2f} {resent:12.0f} {to:9.1f}")
    plt.errorbar([l * 100 for l in losses], med,
                 yerr=[[m - a for m, a in zip(med, lo)], [b - m for m, b in zip(med, hi)]],
                 marker="o", capsize=3, label=f"record {rs} B, M={M}")
plt.yscale("log")
plt.xlabel("Packet loss rate (%)")
plt.ylabel("Throughput (Mbit/s, log scale)")
plt.title("BLAST throughput vs. packet loss rate (median, min-max bars)")
plt.grid(True, which="both", alpha=0.3)
plt.legend(fontsize=8)
plt.tight_layout()
plt.savefig("throughput_vs_loss.png", dpi=150)

# best setting at each loss rate = maximum achievable throughput
best = {}
for key, by_loss in rows.items():
    for l, rs in by_loss.items():
```

```
        m = st.median(tput(rs))
        if l not in best or m > best[l][0]: best[l] = (m, key)
print("\nmaximum throughput per loss rate (best median):")
for l in sorted(best):
    m, (rs, M) = best[l]
    print(f"  loss {l*100:5.1f}%: {m:10.2f} Mbit/s  (record {rs} B, M={M})")

losses = sorted(best)
plt.figure(figsize=(7, 4))
plt.plot([l * 100 for l in losses], [best[l][0] for l in losses], marker="o")
plt.yscale("log")
plt.xlabel("Packet loss rate (%)")
plt.ylabel("Throughput (Mbit/s, log scale)")
plt.title("Maximum BLAST throughput vs. packet loss rate")
plt.grid(True, which="both", alpha=0.3)
plt.tight_layout()
plt.savefig("throughput_max.png", dpi=150)
print("saved throughput_vs_loss.png, throughput_max.png")
```